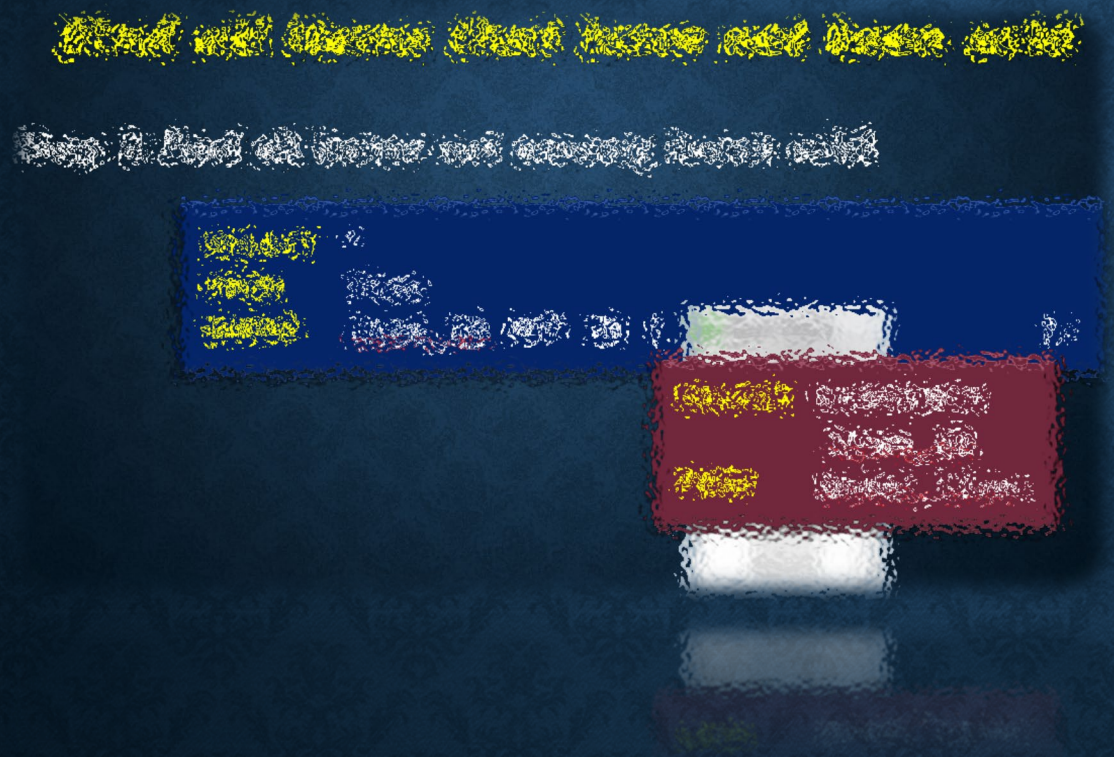
Relational Database & SQL

SQL: Subqueries



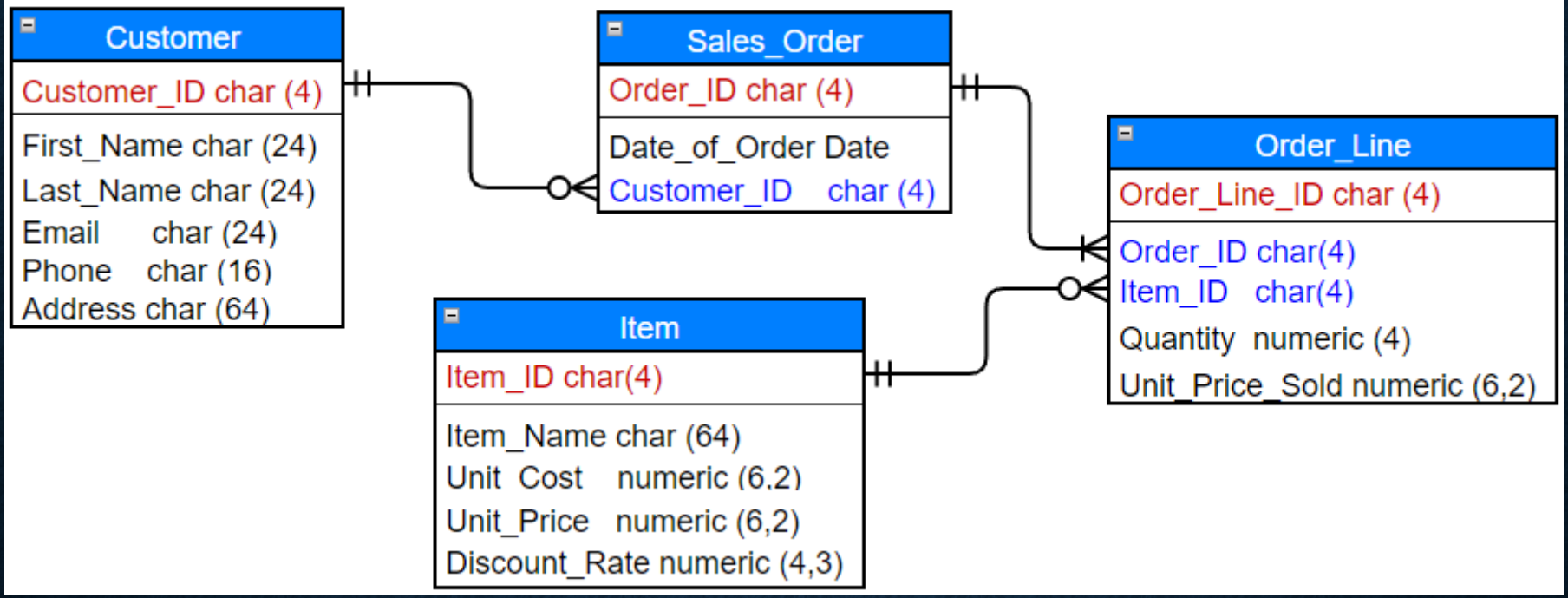
© Copyright Shuguang Hong, Ph.D.
Computer Information Systems, Georgia State University

LEARNING OBJECTIVES

- Study the divide-and-conquer strategy (DaC) to solve complex problems
- Apply the strategy to formulate solutions to complex query problems
- Describe the concept of subqueries to implement the divide-and-conquer solutions in SQL
- Hands-on practices & reflection

Dr. Shuguang Hong, GSU © Copyright

EXERCISE DB: ORDER MANAGEMENT



Dr. Shuguang Hong, GSU © Copyright



WHAT'S DIVIDE-AND-CONQUER?



GenAI

Ask your AI tool to:

- Explain this strategy and use cases in our daily life
- Provide query examples that utilize this strategy (in the Order Management database)

Dr. Shuguang Hong, GSU © Copyright



A USE CASE OF THE DIVIDE-AND-CONQUER STRATEGY





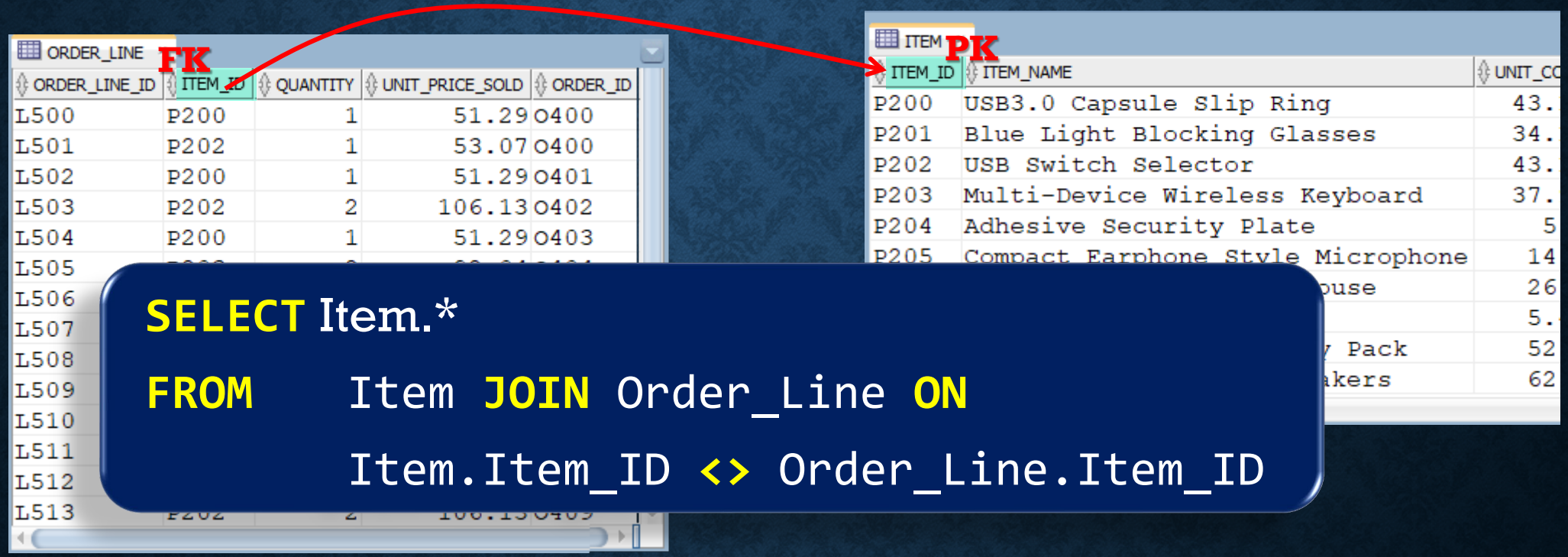
Write a query to find out all items that have not been sold

GenAI

- Work with your AI tool to formulate two solutions to the problem using JOIN and subqueries, respectively
- Compare and contrast those solutions and discuss their pros and cons

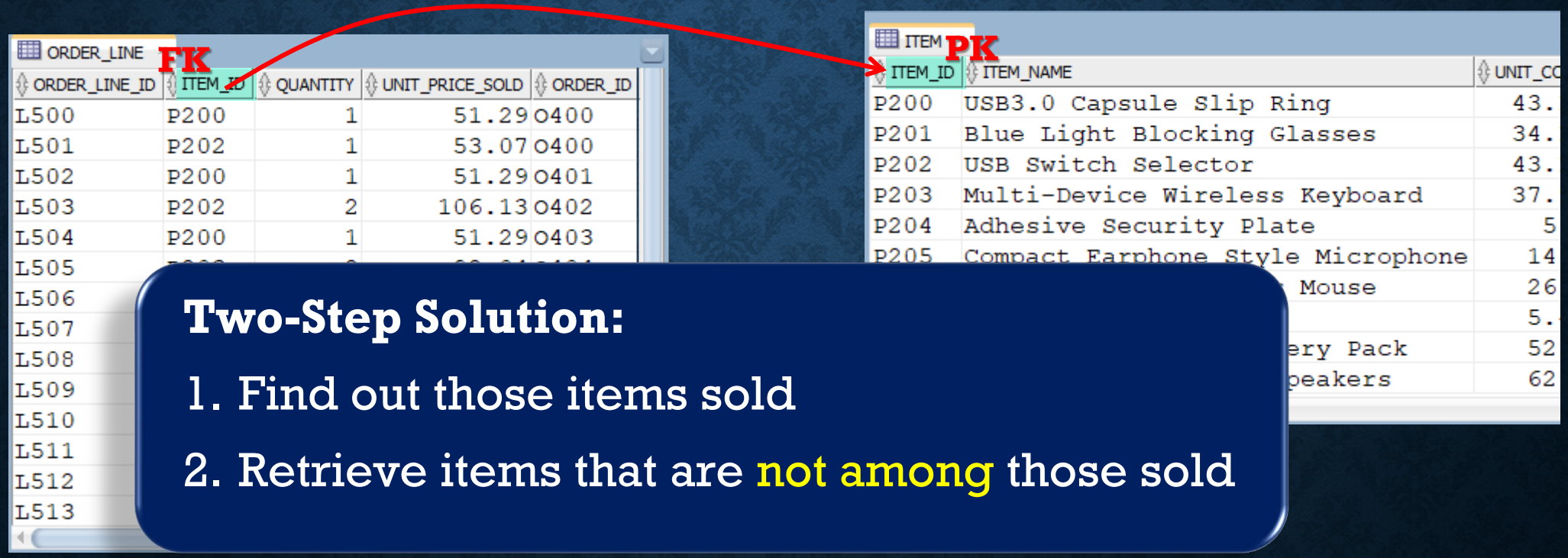
Dr. Shuguang Hong, GSU © Copyright

Find all items that have not been sold



GenAI Ask your AI tool to explain why the SQL query is incorrect

Find all items that have not been sold



Two-Step Solution:

1. Find out those items sold
2. Retrieve items that are not among those sold

Dr. Shuguang Hong, GSU © Copyright

Find all items that have not been sold

Step 1: find all items sold

```
SELECT DISTINCT Item_ID
FROM   Order_Line;
```

ORDER_LINE

ORDER_LINE_ID	ITEM_ID	QUANTITY	UNIT_PRICE_SOLD	ORDER_ID
L500	P200	1	51.29	O400
L501	P202	1	53.07	O400
L502	P200	1	51.29	O401
L503	P202	2	106.13	O402
L504	P200	1	51.29	O403
L505	P203	2	92.04	O404
L506	P200	2	102.58	O405
L507	P202	1	53.07	O405
L508	P206	1	47.49	O406
L509	P200	1	51.29	O407
L510	P203	2	92.04	O408
L511	P205	1	14.28	O408



Query Result

ITEM_ID
P200
P202
P207
P203
P206
P205

Dr. Shuguang Hong, GSU © Copyright

Find all items that have not been sold

Step 2: find all items not among items sold

```
SELECT *
FROM   Item
WHERE  Item_ID NOT IN (
                        );
```

```
SELECT DISTINCT
       Item_ID
FROM   Order_Line;
```


Dr. Shuguang Hong, GSU © Copyright

Find all items that have not been sold

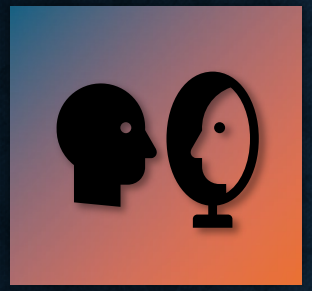
Final solution:

```
SELECT *
FROM   Item
WHERE  Item_ID NOT IN
           (SELECT DISTINCT Item_ID
```

GenAI

- Ask your AI tool for two other complex use cases of the divide-and-conquer strategy
- Test the queries in your local database and work with your AI tool to fix errors if any

Dr. Shuguang Hong, GSU © Copyright



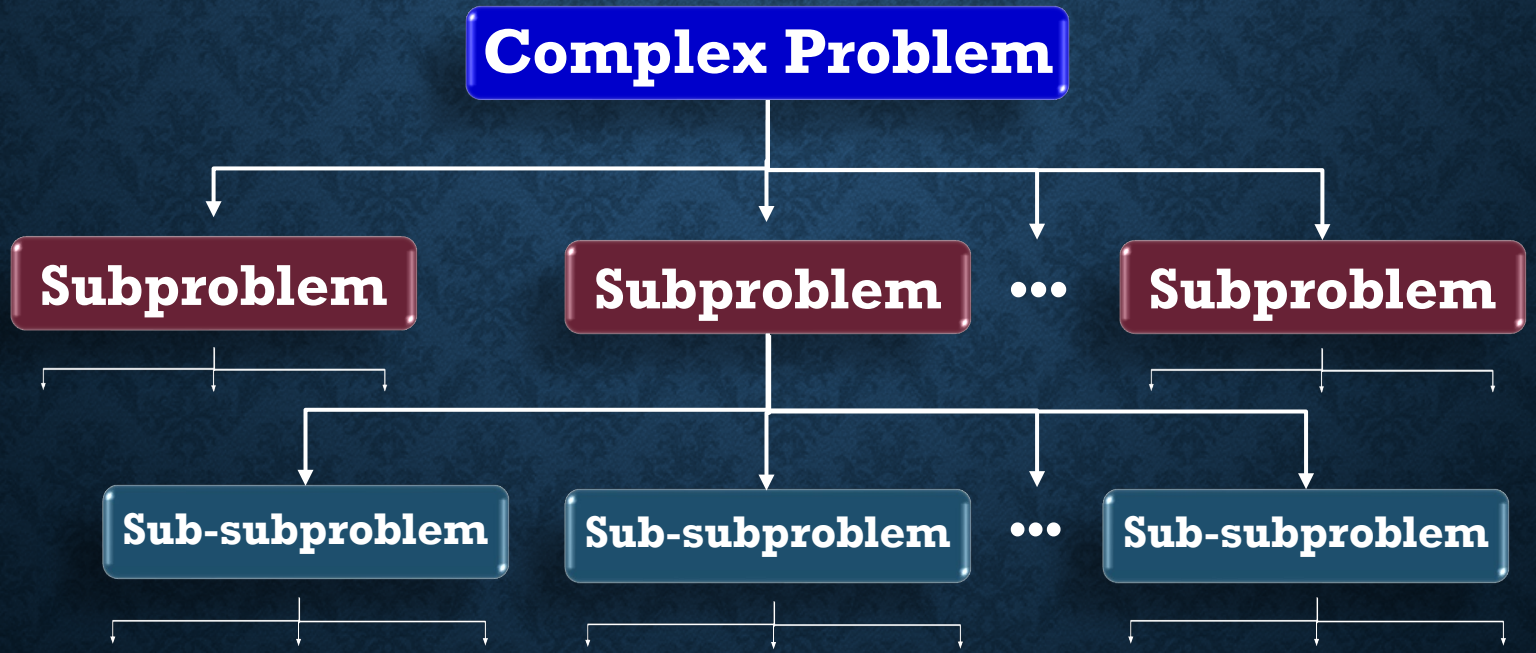
TAKEAWAYS…

- Two phase strategy:
  - Decomposes a complex problem into small sub-problems
  - Construct the final solution by solving each sub-problem, from inside-out (bottom-up)

Dr. Shuguang Hong, GSU © Copyright

Takeaways…

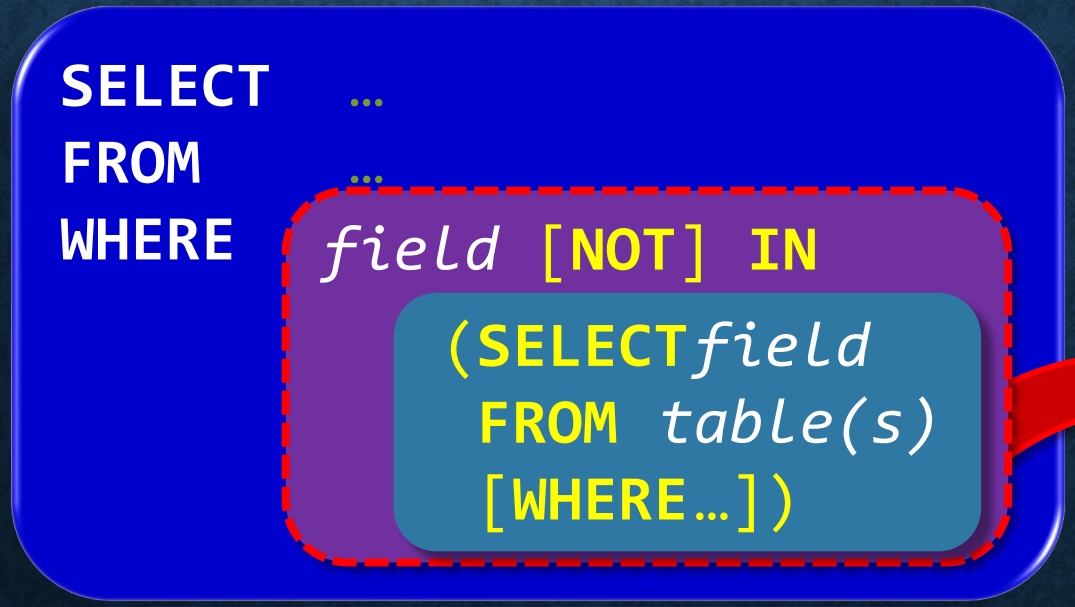
THE DIVIDE-AND-CONQUER STRATEGY ILLUSTRATED



⚠ *Its usefulness can be diminished quickly if it's overdone*

Dr. Shuguang Hong, GSU © Copyright

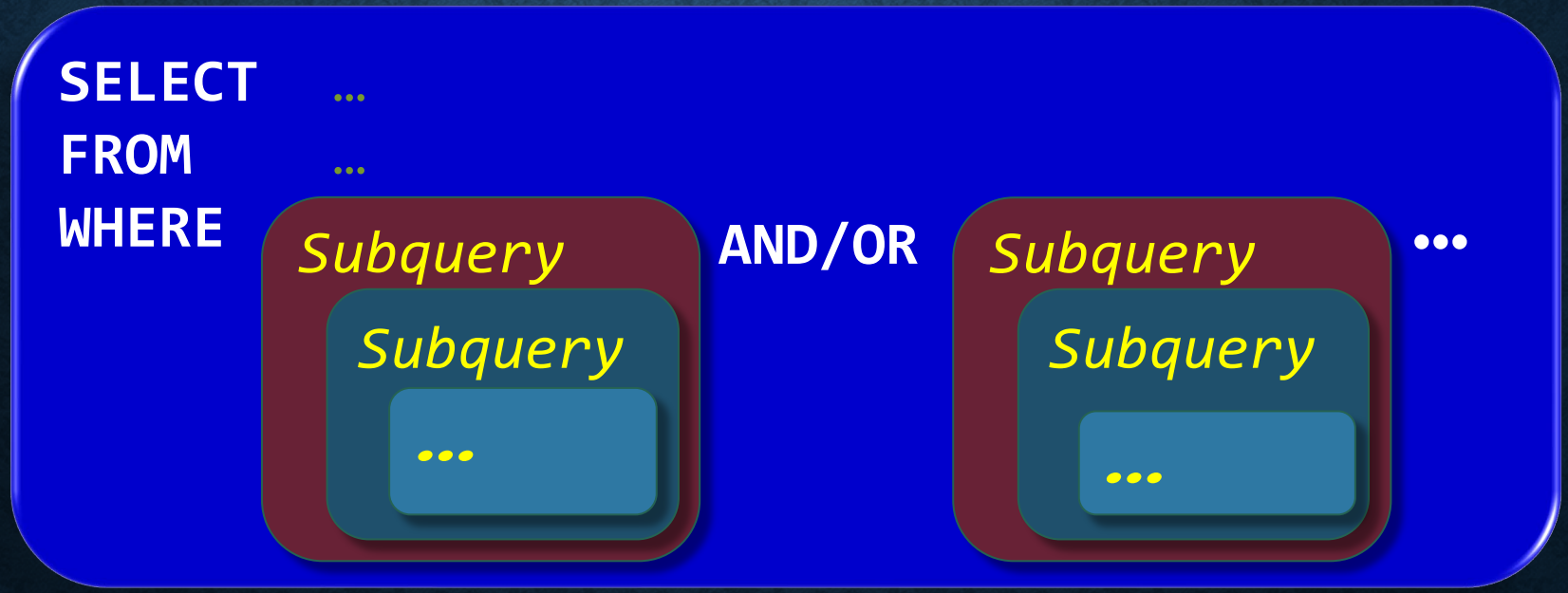
ONE-LEVEL SUBQUERY (NESTED QUERIES) SYNTAX



The inner query must be linked to a field in the outer query's WHERE clause by a Boolean expression

Dr. Shuguang Hong, GSU © Copyright

A COMPLEX, NESTED SUBQUERIES ILLUSTRATION



Dr. Shuguang Hong, GSU © Copyright

THE HAVING CLAUSE MAY USE SUBQUERIES AS GROUP FILTERS

```
SELECT    …
FROM      …
WHERE     …
GROUP BY  …
HAVING field [NOT] IN
        (SELECT field
         FROM   table(s)
         [WHERE …])
```

The subquery and linked field must form a Boolean expression as a group filter

Dr. Shuguang Hong, GSU © Copyright



A CHALLENGE



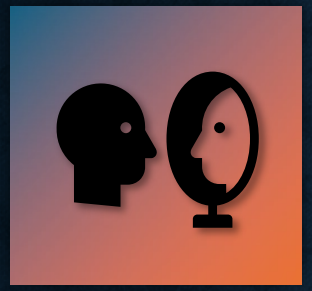


Find the most profitable items, i.e., items with the largest total sales amount

GenAI

- Work with your AI tool to build a query solution using subqueries and explain the steps
- Ask your AI tool for a solution which does not use any subquery
- Compare & contrast those two solutions
- Test the solutions in your database

Dr. Shuguang Hong, GSU © Copyright



TAKEAWAYS…

- Divide-and-conquer approach helps construct a solution to a complex problem, one step at a time
  - A bottom-up, inside-out approach
- A subquery may be nested inside another subquery
- A subquery may be used to
  - Select specific rows in the WHERE clause
  - Select specific groups in the HAVING clause
- Practice makes perfect

Dr. Shuguang Hong, GSU © Copyright